

# AccessLayer

## Product Requirements Document (PRD)

### AI-Powered Accessible Learning Platform for Blind Students

Hackathon MVP — 48 Hours

**One-line pitch:** One course. One upload. An AI-powered learning experience designed to make university education accessible to blind students.

## 1. Product Vision

AccessLayer is an AI-powered university learning platform designed around blind students as the primary user. Teachers upload course material once; AccessLayer processes it and provides a keyboard-first, screen-reader-friendly learning experience while AI makes course content easier to navigate, understand, and interact with.

## 2. Problem

University course material often contains PDFs, lecture notes, images, diagrams, tables, assignments, quizzes, and announcements. Blind students may encounter difficulties when material is poorly structured, visually dependent, difficult to navigate with a keyboard, missing meaningful descriptions, or presented in formats that do not work well with assistive technology.

**Core problem:** How can one piece of university content become easier for a blind student to navigate and understand without requiring the teacher to recreate everything?

## 3. Target Users

**Primary — Blind Student:** navigate without a mouse, hear interface feedback, access course material, understand visual content, complete quizzes, and ask questions about course material.

**Secondary — Teacher:** upload course material once, create assignments/quizzes, publish announcements, and see whether content has accessibility problems.

## 4. MVP Scope

**Student:** keyboard-first navigation; TTS announcements; courses; notes/materials; assignments; quizzes; announcements; AI course assistant; AI-generated descriptions of selected visual content; RAG over uploaded course material.

**Teacher:** upload PDF/course material; create basic announcement; create basic quiz; see processing status.

**AI:** document understanding; content extraction; RAG; image/diagram description; course Q&A; basic accessibility analysis.

## 5. Explicitly Out of MVP

Full university management system; voice-based form filling; full voice-controlled navigation; mobile application; multiple accessibility modes; support for every disability; university ERP/LMS integration; real authentication system; complex grading; live classroom; custom AI model training; full WCAG certification/compliance; production infrastructure.

## 6. Core User Journey

**Teacher:** Login → Teacher Dashboard → Create Course → Upload PDF → AI Processes Material → Accessibility Analysis → Course Published.

**Student:** Login → TTS welcome → Course List → Select Course → Notes / Assignments / Quizzes / Announcements / Ask AI.

## 7. Keyboard Navigation

Core interaction is keyboard-first. Example course menu: 1 Notes, 2 Assignments, 3 Quizzes, 4 Announcements, 5 Ask AI, 0 Back. Selecting an option triggers a spoken confirmation.

## 8. Text-to-Speech

TTS announces important interface changes such as login success, course selection, unread announcements, quiz progress, selected options, and AI responses. Avoid speaking everything automatically; use TTS for meaningful navigation and status information.

## 9. AI Course Assistant

The assistant answers questions using course-provided material. Example queries: “What assignments do I have?”, “Explain the first lecture,” and “What is backpropagation?” Answers should prioritize uploaded course content rather than turning the platform into a generic chatbot.

## 10. RAG Architecture

Teacher uploads PDF → extraction → chunking → embeddings → vector database → retriever → LLM → answer with source/page reference. If the information is not in the course material, the assistant should say so rather than inventing an answer.

## 11. AI Visual Understanding

When a lecture contains a diagram, chart, or image, AI identifies the visual content and generates a structured accessible description. For example, a neural-network diagram can be converted into an explanation of its input layer, hidden layers, output layer, and data flow, which can then be read aloud.

## 12. Accessible Quiz

Quizzes should be fully keyboard navigable. Questions and answer choices are announced by TTS; students select an option using a number and Enter; the system provides feedback and announces quiz progress.

## 13. Accessibility Analysis

After upload, AI reports detected accessibility issues such as images without useful descriptions, diagrams requiring visual interpretation, tables that may be difficult for screen readers, and inconsistent structure. Provide an “Auto-Fix” concept for generating accessible alternatives.

## 14. System Architecture

Streamlit UI → Teacher/Student UI → Python backend → PDF parser + AI engine + database → RAG engine + vision model → AI response → TTS.

## 15. Suggested Tech Stack

**Frontend:** Streamlit

**Backend:** Python

**LLM:** reliable hackathon-provided API (e.g. Gemini, Groq, OpenAI)

**PDF:** PyMuPDF

**RAG:** ChromaDB or FAISS

**Embeddings:** available embedding model/API

**Vision:** multimodal model

**TTS:** browser Web Speech API for MVP

**Storage:** SQLite or JSON

## 16. Data Model

User: id, name, role, accessibility\_mode.

Course: id, name, description, teacher\_id.

Document: id, course\_id, filename, processed.

Assignment: id, course\_id, title, deadline.

Quiz: id, course\_id, questions.

Announcement: id, course\_id, content.

## 17. AI Components

**Content Analyzer:** understands document structure, extracts content, identifies images/tables/diagrams.

**Accessibility Agent:** identifies accessibility problems and generates accessible representations.

**Learning Assistant:** retrieves course content, reasons over it, answers student questions, and provides sources.

## 18. Killer Demo

Start with a teacher uploading a difficult lecture PDF. Show the AI analyzing the document and identifying visual accessibility issues. Generate an accessible representation. Switch to blind-student mode and navigate entirely by keyboard. Ask the AI to explain a diagram and answer a course question using RAG. This demonstrates accessibility + AI + RAG + multimodal understanding.

## 19. Judge Positioning

Do not pitch it as “another LMS.” Pitch it as an **AI accessibility layer for education**: teacher creates content once → AI analyzes it → detects accessibility barriers → generates accessible representations → blind student receives an adapted learning experience.

## 20. Success Criteria

Keyboard-only navigation works; important interface information is spoken; a student can complete a quiz using keyboard; the AI answers course questions from uploaded material; visual educational content can receive AI-generated descriptions; teacher can upload one document and receive an accessibility analysis; demo runs without manual backstage intervention.

## 21. 48-Hour Development Priority

**P0 — Must work:** Streamlit app, teacher upload, PDF extraction, course storage, student navigation, keyboard interaction, TTS, RAG Q&A;

**P1 — Should work:** vision/diagram description, quiz, accessibility audit, source/page references.

**P2 — Nice to have:** voice commands, advanced analytics, advanced teacher dashboard, automatic scoring, additional formats. Delete P2 if time runs short.

## 22. Biggest Technical Risks

**PDF extraction:** prepare reliable demo PDFs.

**TTS failure:** browser TTS plus visible text fallback.

**Hallucination:** RAG, source references, and explicit “not found” behavior.

**Keyboard issues:** keep UI simple.

**Vision latency:** process only pages containing relevant visual content.

## 23. Future Vision

Voice navigation; voice form filling; advanced diagram interaction; OCR for scanned documents; dyslexia/cognitive accessibility modes; Urdu accessibility; integrations with Moodle, Canvas, Google Classroom, and university LMS platforms.

## 24. Final Product Positioning

**Product:** AccessLayer

**Preferred tagline:** Teach once. AI makes it accessible.

**Overall assessment:** Buildable in 48 hours if tightly scoped. The accessible LMS itself is not highly novel; the differentiator is the AI accessibility/transformation layer combining document understanding, visual understanding, RAG, and keyboard-first access.

## Hackathon Recommendation

| Dimension             | Assessment                                              |
|-----------------------|---------------------------------------------------------|
| Buildable in 48 hours | Yes — with strict scope                                 |
| Worth building        | Yes — with the AI accessibility-layer positioning       |
| Novelty               | Moderate; stronger when focused on AI transformation    |
| AI relevance          | Strong with RAG + vision + accessibility transformation |
| Risk                  | Moderate                                                |
| Judge potential       | 8–9/10 with a polished, reliable demo                   |